

NLP

RNN LSTM y GRU

Repaso:

Clase 1

- Preprocesamiento: lematizacion, stop-words, stemmings
 - Tecnicas de conteo (no de semantica) para vectorizar documentos (tokenizacion).
- No se tenia en cuenta el orden de las palabras, la semantica, sinonimos podian no tener relacion alguna

Clase 2

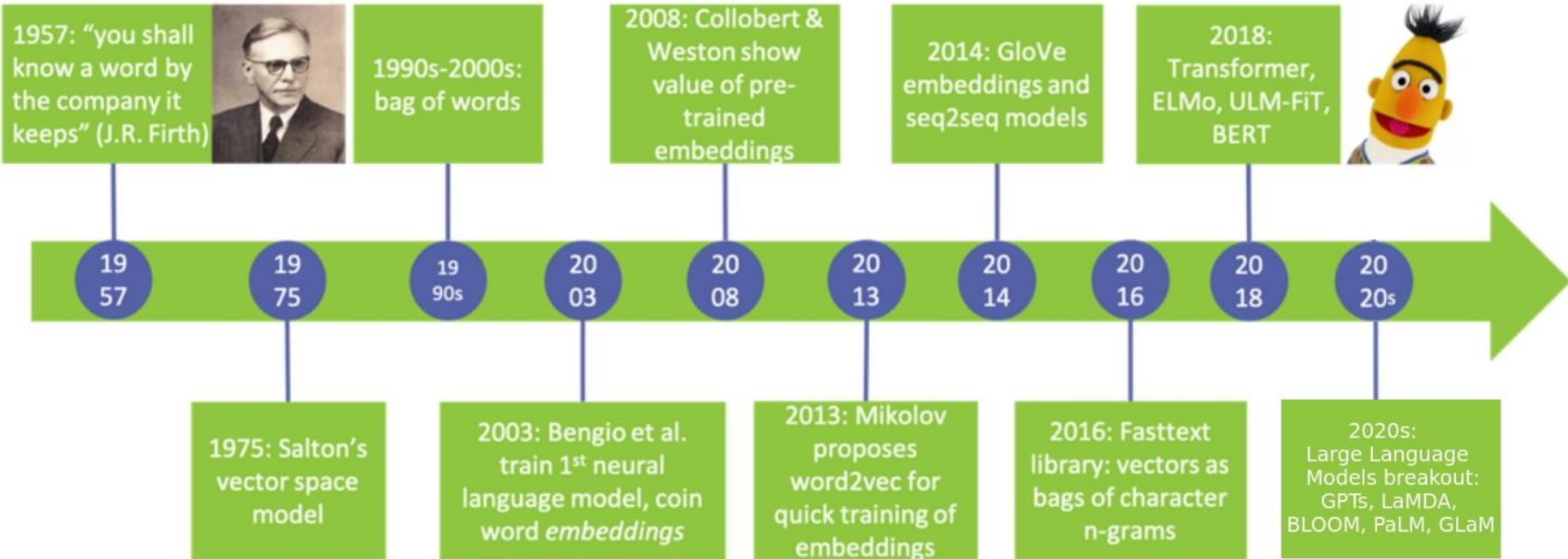
- Embedding: permite convertir textos a numeros. Captura el contenido semantico del lenguaje
- Framewords: Word2Vec, Glove, FastText. Dos formas de crear embedding: CBoW y Skip-gram que definen que es una "palabra" en base a su contexto. De esta forma capturan el orden ya que la ventana se mueve y se puede ver la proximidad de las palabras

Docente:
Josselyn Ordóñez

Timeline



1990: Celda RNN básica (Elman)
1997: LSTM



2016: LSTMs se convierten en SotA para traducción automática

Redes Neuronales Recurrentes (RNNs) - Celda de Elman



Es un tipo de neurona con un estado interno (o memoria) de manera que la información del pasado influye en los resultados futuros.



Se utiliza principalmente para resolver problemas de secuencia, en donde el valor anterior está relacionado con el valor futuro.

No solo se usa en texto, se utiliza donde el orden importa!



Permite construir modelos cuyos tamaños de secuencia sean potencialmente arbitrarios.



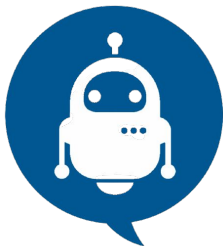
Implementa modelos de lenguaje de la forma:

$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

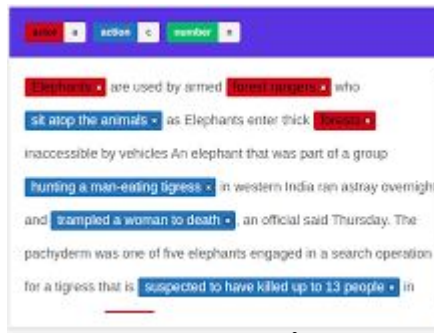
Dado ciertos sucesos pasados, cual es la probabilidad de que un suceso nuevo ocurra

"Hoy el día está hermoso y despejado, se puede ver un hermoso cielo... azul"

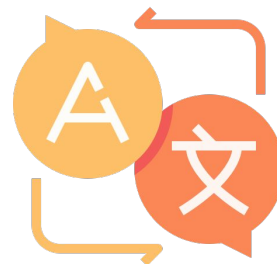
Algunos problemas de secuencia



Bots
Conversacionales



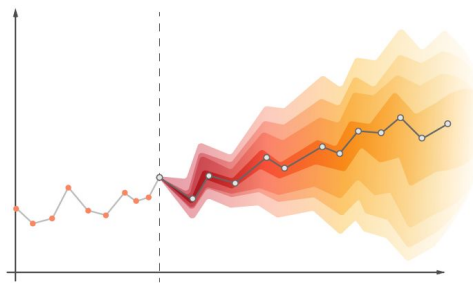
Name entity
recognition



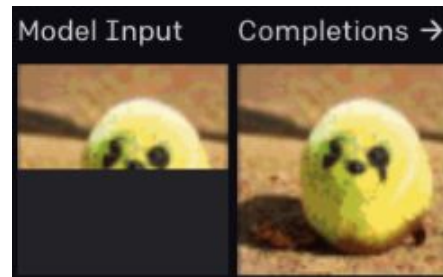
Traducción de
idiomas



Speech to text



Series
temporales



Completar una
imagen

Celda RNN básica (Elman)

[LINK](#)

[API KERAS](#)

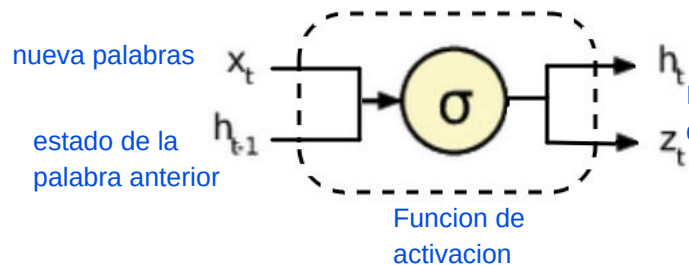


En Keras: SimpleRNN

$$h_t = \sigma(W_{hh} * h_{t-1} + W_{hx} * x + b_h)$$

$$z_t = h_t$$

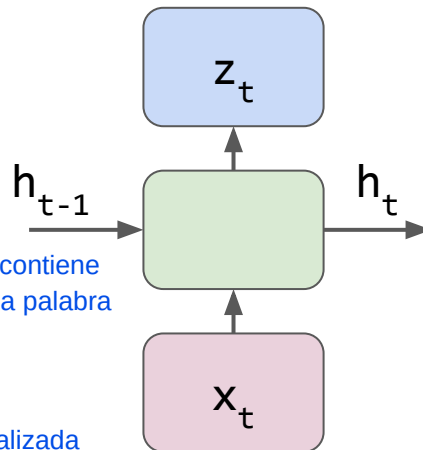
RNN Unit



Estado interno: contiene informacion de la palabra anterior

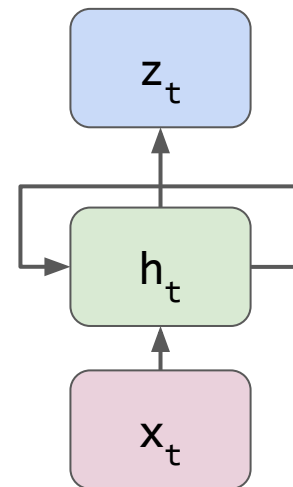
Informacion actualizada de la oracion

Output: convina la informacion de las palabras pasadas mas la informacion actual



Unidad básica

Esta celda se repite por cada palabra



Representación compacta

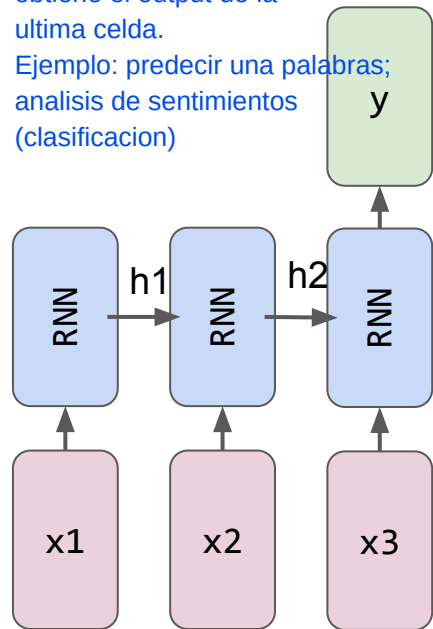
Tipos de problemas de secuencia

[LINK](#)



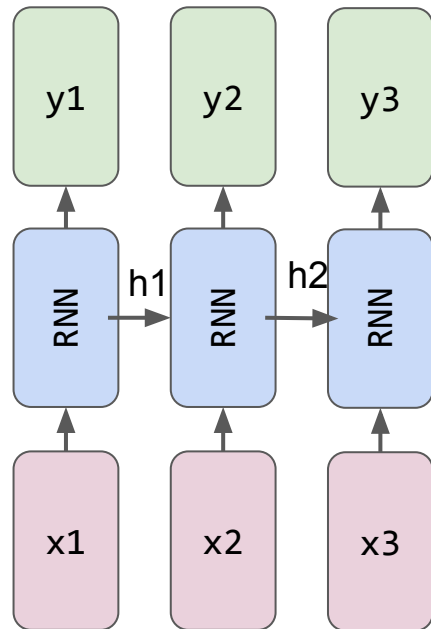
Se procesa todo (la oración) y se obtiene el output de la última celda.

Ejemplo: predecir una palabras;
análisis de sentimientos
(clasificación)



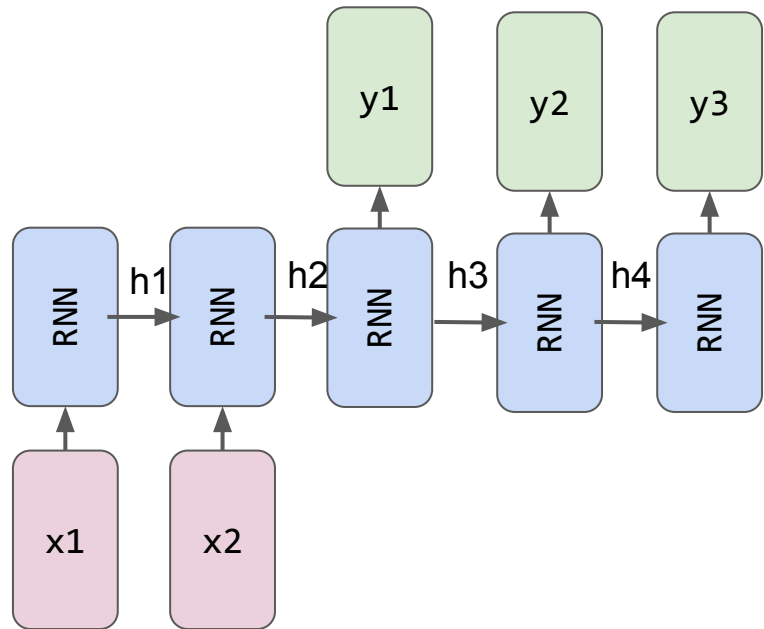
many-to-one

No espera a que procese todo, se obtiene una salida por cada celda.
Ejemplo: reconocimiento de entidad.



many-to-many

Se procesa todo y luego se generan las salidas
Ejemplo: traductor; chatbot conversacionales



seq2seq

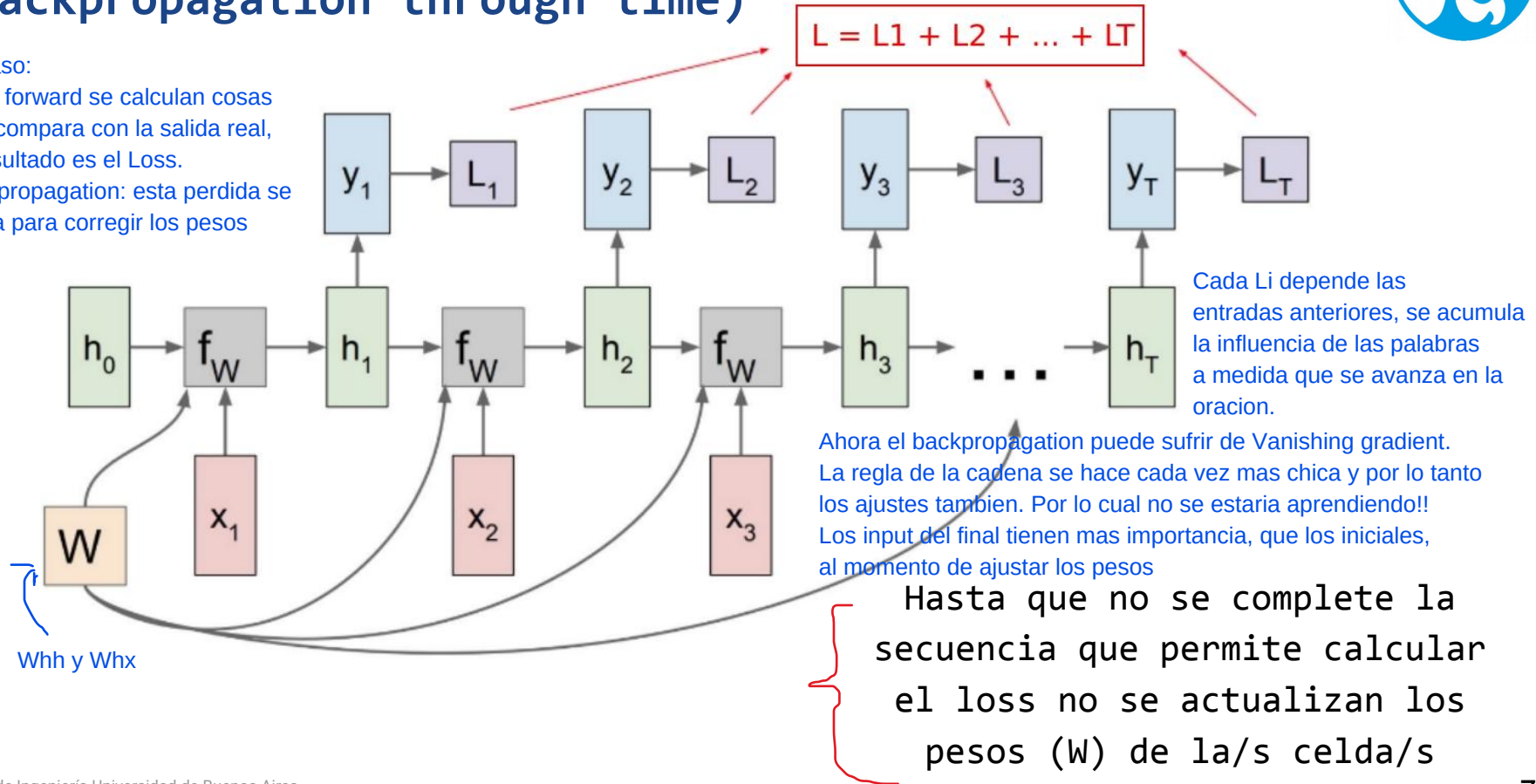


Grafo de cómputo de una RNN y BPTT (Backpropagation through time)

Repaso:

En el forward se calculan cosas y se compara con la salida real, el resultado es el Loss.

Backpropagation: esta perdida se utiliza para corregir los pesos



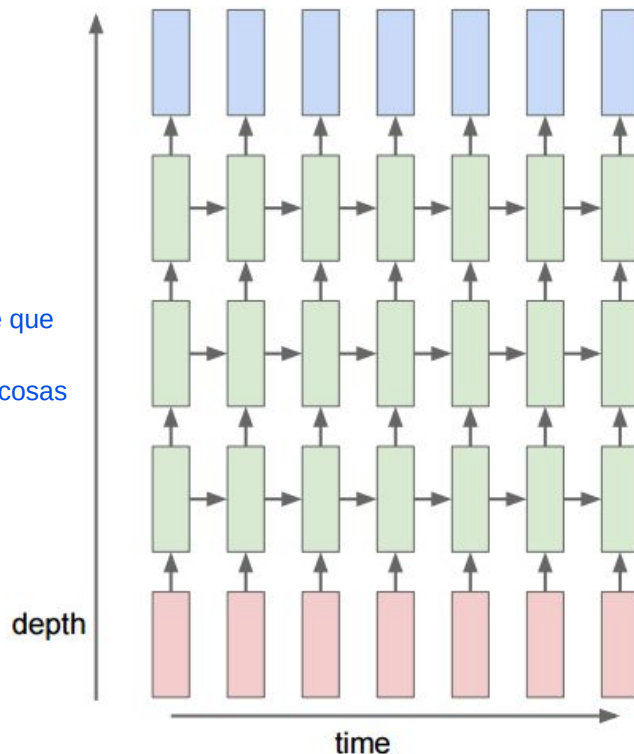
Multi-layer RNN



Tal como se vio en los ejemplos se trata de apilar layers RNN en donde la salida de una se traslada a la entrada de la siguiente

Este tipo de red se utiliza para capturar conceptos mas abstractos.

Por ejemplo en texto, primero se aprende que es un articulo, que es un verbo, que es el amor, etc. Las primeras capas aprenden cosas basicas, y las mas profundas cosas mas abstractas.



El Vanishing Gradient, para una red RNN, se da tanto en la profundidad (N capas) como en el tiempo

Problema de una RNN tradicional

[LINK](#)



"Una RNN tradicional solo usa información del pasado y no de las futuras palabras para predecir"

Ejemplo: Data una sentencia determinar si existe una entidad que represente al nombre de una persona utilizando (name entity recognition)

Ejemplo 1:

"Hoy escuche que *Victoria* terminó su bot para NLP" → persona

Ejemplo 2:

"Hoy escuche que *Victoria* cambió de intendente" → ciudad

La
contextualización
de la palabra es
a futuro

Para solventar este problema se crearon las Bidireccional RNN

Bidirectional RNN (BRNN)

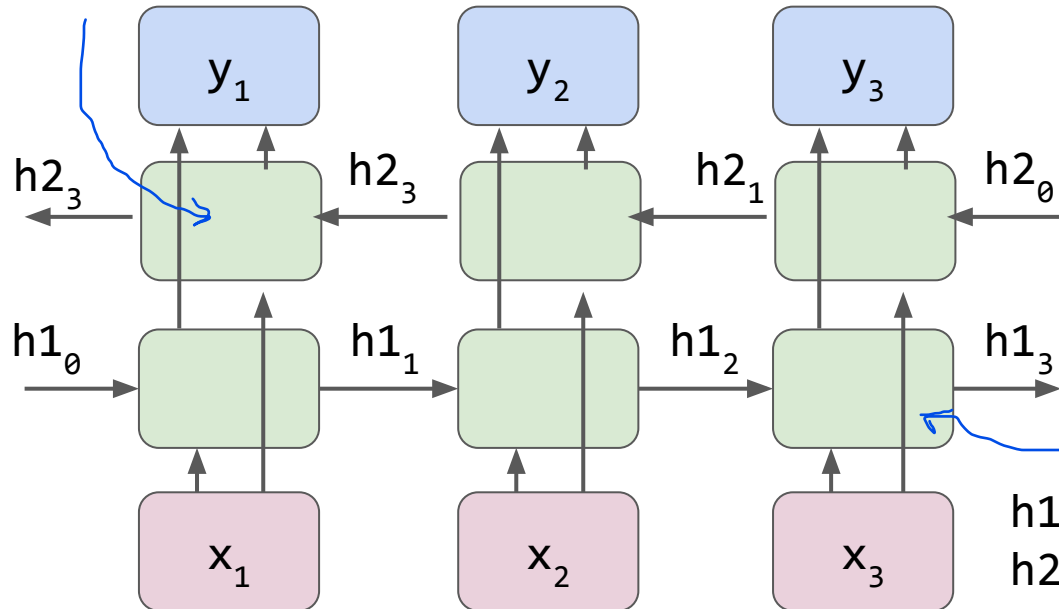
[BRNN PAPER](#)

[API KERAS](#)



“La palabra anterior y la palabra futura tienen impacto en la presente predicción”

X1 se procesa con información del futuro



Ambas capas se procesan en paralelo.
Cada entrada se procesa dos veces

X3 se procesa con información del pasado

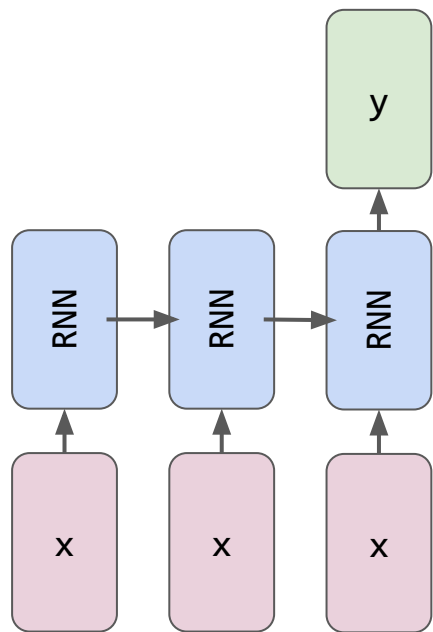
$$h1_t = \sigma(W_{hh1} * h1_{t-1} + W_{hx1} * x + b_1)$$
$$h2_t = \sigma(W_{hh2} * h2_{t+1} + W_{hx2} * x + b_2)$$

Las dos salidas pueden concatenarse, sumarse o promediarse

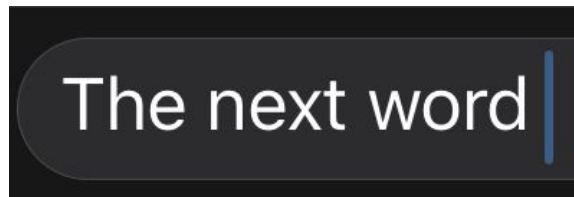
many-to-one



"Dada una sentencia o oración de entrada de tamaño fijo, el sistema arroja un único resultado que la representa".



Este tipo de estructuras se utilizan para determinar cuál es la siguiente palabra o elemento en la secuencia o para clasificación (sentiment analysis).



Predicción de
próxima palabra



Análisis de
sentimientos

many-to-one

[Nota: ver ejemplo en notebook](#)

Long short term memory (LSTM)

[LINK](#)



Se introduce este tipo de celda neuronal con mayor persistencia de memoria para lograr capturar relaciones de palabras a largo plazo.

Evita el problema de Vanishing Gradient, no es afectada por el Backpropagation



Se crearon en 1997. Se adoptó como la layer principal para problemas de secuencia en 2014 hasta la aparición de los transformers en 2017.



Desplazaron completamente a las capas RNN simples (Elman), ya que el costo adicional de las LSTM es marginal respecto al beneficio que otorgan



Se basan en el principio de ponderar la importancia de una palabra respecto al contexto futuro/pasado (key words).

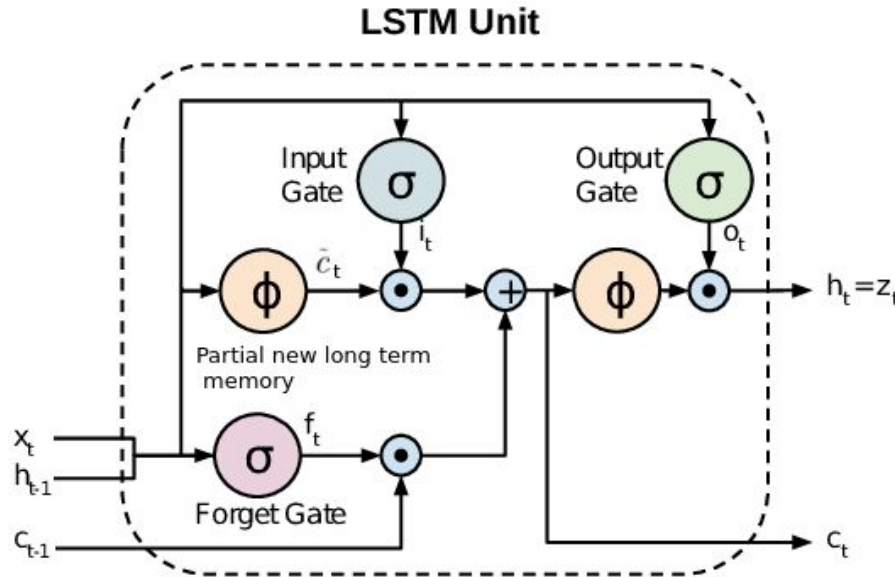
¿Comprarías este producto?

LSTM incorpora un concepto similar al Skip-connection (ResNet), que es la memoria de largo plazo.

*"**Incredible!** El producto es lo que venden, hace lo que tiene que hacer y me **ayudó mucho** a resolver los problemas que tenía. Lo **volvería a comprar** sin dudas"*

Con una SimpleRNN es posible que las palabras del inicio se pierdan

LSTM approach - Idea de la arquitectura



C: 'Memoria a largo plazo'

Input (i): Dado el último estado (h_{t-1}) evalúa cuánto de la nueva entrada (x) se incluirá en la memoria de largo plazo (C_t).

Forget (f): Dada la entrada (X) cuánto del estado de memoria anterior (C_{t-1}) tiene importancia en el nuevo estado. Cuanto de la información pasada se quiere olvidar/recordar

Output (o): Qué parte del estado de memoria (C_t) pasa al próximo estado de memoria h_t .

$\cdot$: Producto elemento a elemento

ϕ : Función Tanh

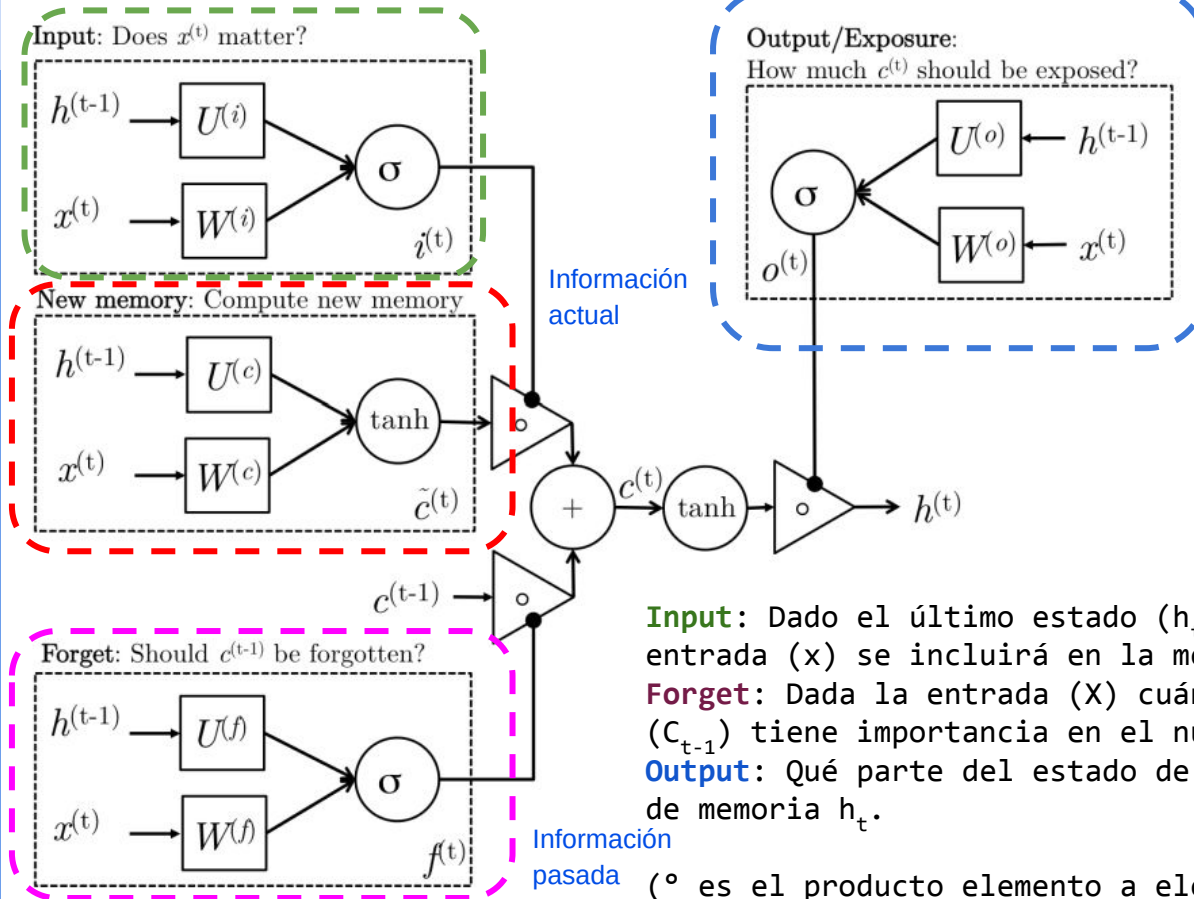
$+$: Suma vectorial

$\rightarrow$: Conexiones

LSTM approach

[LINK](#)

En LSTM ahora hay 4 celdas con U y W pesos entrenables, por lo tanto es mas grande que una RNN



$$i_t = \sigma(W^{(i)}x_t + U^{(i)}h_{t-1})$$

$$f_t = \sigma(W^{(f)}x_t + U^{(f)}h_{t-1})$$

$$o_t = \sigma(W^{(o)}x_t + U^{(o)}h_{t-1})$$

$$\tilde{c}_t = \tanh(W^{(c)}x_t + U^{(c)}h_{t-1})$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$$

$$h_t = o_t \circ \tanh(c_t)$$

Input: Dado el último estado (h_{t-1}) evalúa cuánto de la nueva entrada (x) se incluirá en la memoria de largo plazo (C_t).

Forget: Dada la entrada (x) cuánto del estado de memoria anterior (C_{t-1}) tiene importancia en el nuevo estado.

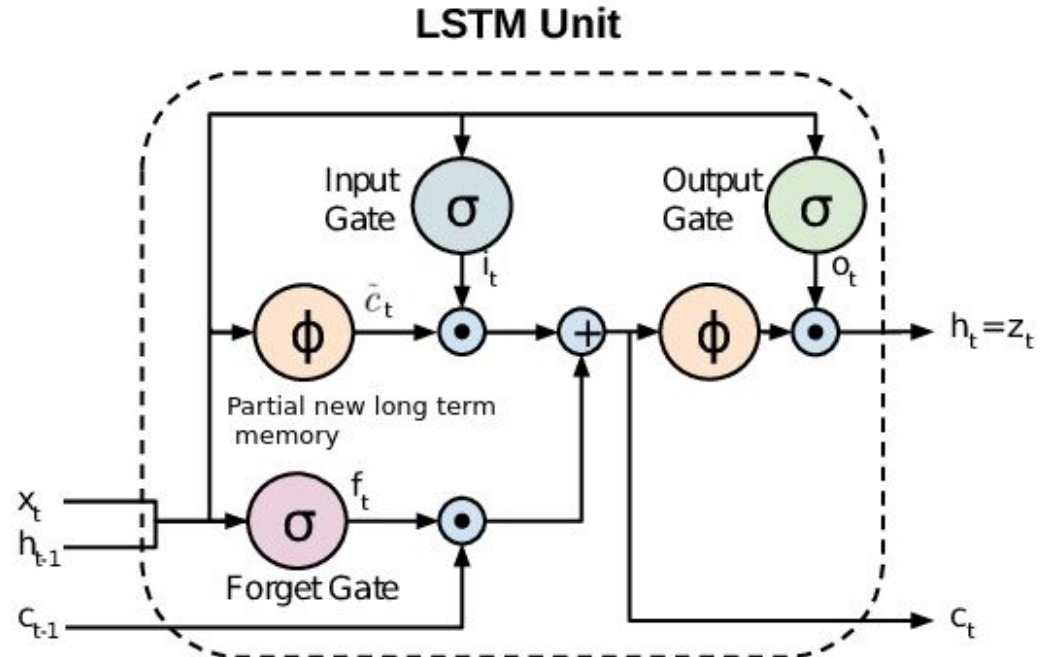
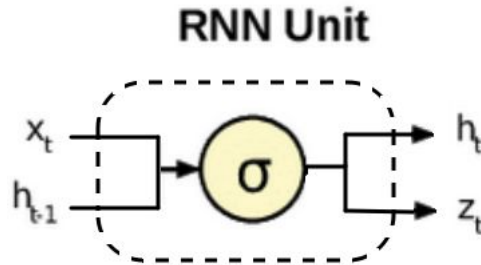
Output: Qué parte del estado de memoria (C_t) pasa al próximo estado de memoria h_t .

($\circ$ es el producto elemento a elemento)

LSTM vs RNN



La memoria de largo plazo c_t permite propagar gradiente eficientemente a mayor “profundidad temporal”.



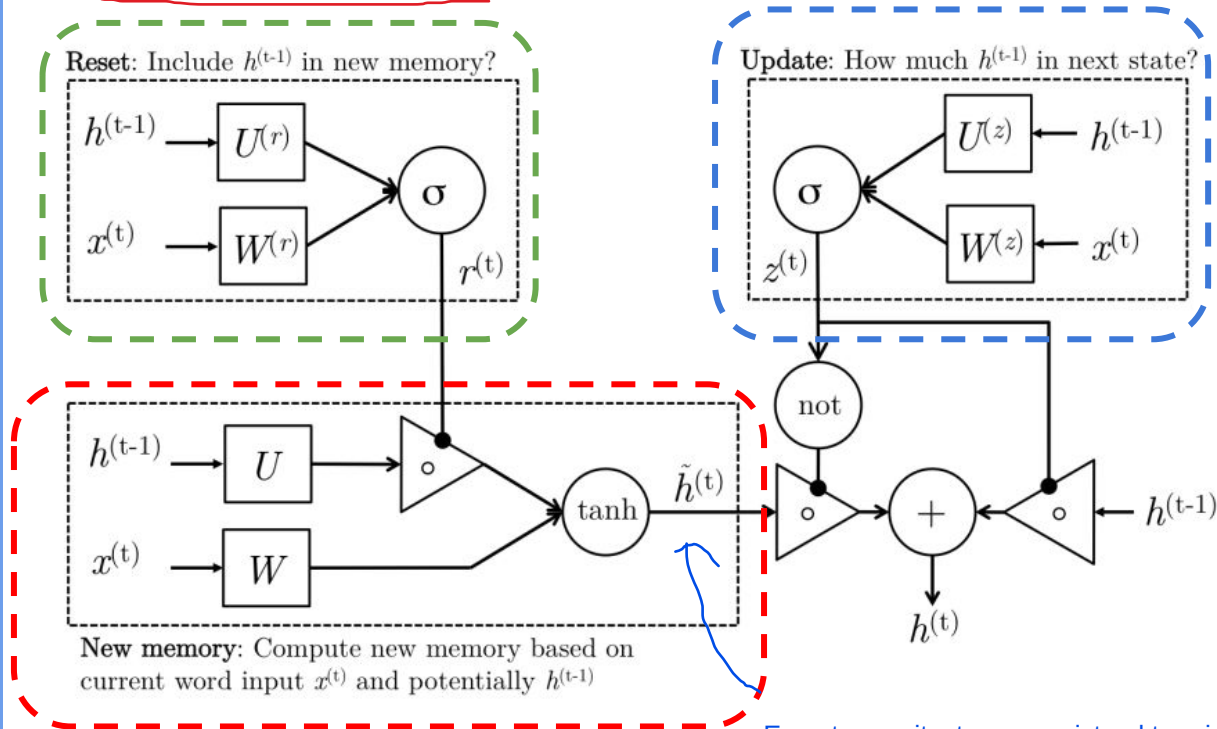
NOTA: c_{t-1} pasa directo a la salida c_t , es como una skip-connection. Con esto se ahorra el procesamiento de una capa de activación. La memoria C se transmite mejor de capa en capa

Gated Recurrent Units (GRU)

[LINK](#)



"Evolución de las RNN para superar problemas de "short-memory", versión reducida de una LSTM". Usa el mismo concepto de transmitir la memoria pero mas eficiente (menos pesos entrenables)



Esta celda se usa tanto para la información presente como la pasada (negando)

$$z_t = \sigma(W^{(z)}x_t + U^{(z)}h_{t-1})$$

$$r_t = \sigma(W^{(r)}x_t + U^{(r)}h_{t-1})$$

$$\tilde{h}_t = \tanh(r_t \circ U h_{t-1} + W x_t)$$

$$h_t = (1 - z_t) \circ \tilde{h}_t + z_t \circ h_{t-1}$$

(Update gate)

(Reset gate)

(New memory)

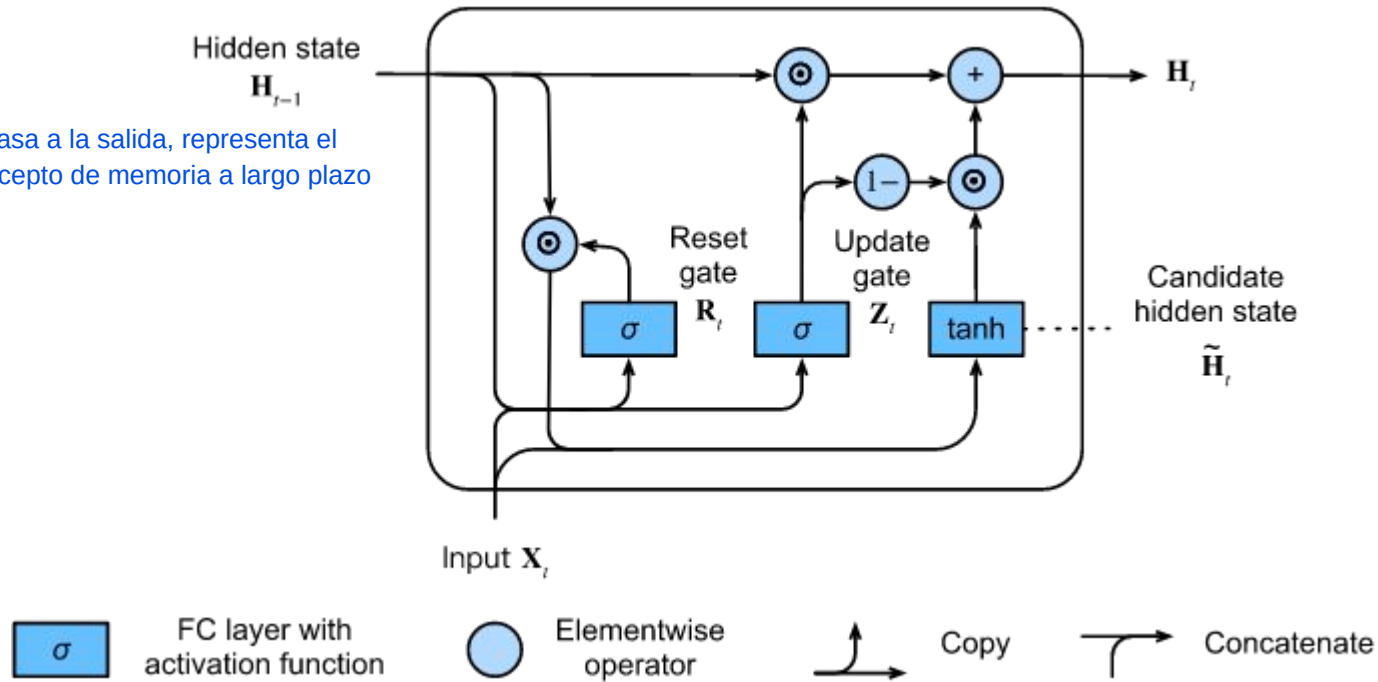
(Hidden state)

En esta arquitectura no existe el termino C, ahora h (estado interno) es la memoria, Es decir, no hay un termino diferente para la memoria

Gated Recurrent Units (GRU)



H pasa a la salida, representa el concepto de memoria a largo plazo



Predicción de texto - Modelos de lenguaje



Objetivo: Entender la estructura estadística del lenguaje aprovechando su estructura secuencial.

Tarea: Aprender a predecir la siguiente palabra.

$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

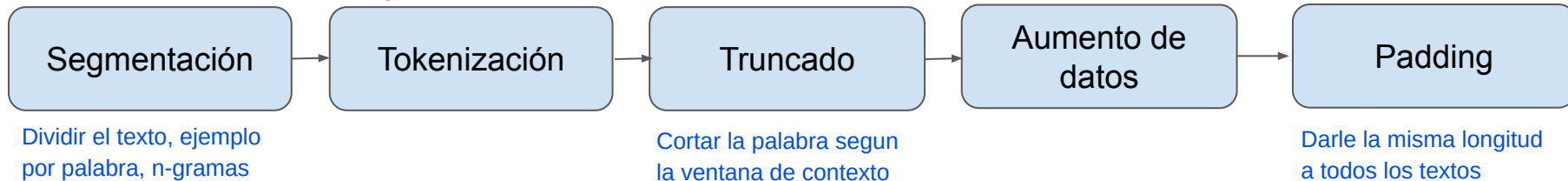
The next word |

$$(X_1, \dots, X_n) \rightarrow X_{n+1}$$

Pasos a seguir:

Asignar un numero (id) a los segmentos. Si la palabra aparece varias veces, se asigna el mismo numero!

Generar mas datos en base al texto dado



Segmentación y Tokenización



Documentos:

`'Yesterday, all my troubles seemed so far away'`

Segmentación:

En este ejemplos es segmentacion por palabra

`['yesterday', 'all', 'my', 'troubles', 'seemed', 'so', 'far', 'away']`

Tokenizacion:

→ `[ 34, 189, 200, 1345, 8, 69, 12,753]`

Ventana de contexto



La **ventana de contexto** en un modelo de lenguaje es la cantidad máxima de tokens (palabras o subpalabras) que el modelo puede considerar a la vez como entrada. Define el alcance del "pasado" que el modelo puede usar para predecir la siguiente palabra o generar texto.

Si la secuencia es más **CORTA**
que la ventana de contexto



Padding (relleno)

Si la secuencia es más **LARGA**
que la ventana de contexto



Truncado

Por ejemplo, en un LLM si se quiere procesar un PDF que excede la ventana,
entonces se trunca

Truncado de secuencias:



Si tenemos una secuencia de tamaño M y una ventana de contexto de tamaño N tal que $M > N$, entonces podemos generar $1 + M - N$ secuencias de tamaño N :

Ejemplo ($M=6$, $N=3$)

M : tamaño de la secuencia

N : tamaño de la ventana de contexto

["Todas", "las", "hojas", "son", "del", "viento"] se convierte en:

["Todas", "las", "hojas"]

["las", "hojas", "son"]

["hojas", "son", "del"]

["son", "del", "viento"]

Data Augmentation y padding



Queremos que el modelo sea capaz de predecir cada elemento de la secuencia, no solo el último:

["Todas", "las", "hojas", "son", "del", "viento"] → [1, 2, 3, 4, 5, 6]

Ejemplo (M= 6, N=3)

El 0 representa un token
sin informacion

["Todas", "las", "hojas"]
["las", "hojas", "son"]
["hojas", "son", "del"]
["son", "del", "viento"]

Data augmentation

["Todas"] -> [0, 0, 1]
["Todas", "las"] -> [0, 1, 2]
["Todas", "las", "hojas"] -> [1, 2, 3]

El padding se agrega al inicio porque, en general,
se quiere predecir la siguiente palabra y ésta
tiene que estar al lado de la ultima.

Probabilidad de una secuencia



$$\begin{aligned} P_{LM} (x^{T+1}, x^T, \dots, x^1) &= P_{LM} (x^{T+1} | x^T, \dots, x^1) P_{LM} (x^T, x^{T-1}, \dots, x^1) \\ &= P_{LM} (x^{T+1} | x^T, \dots, x^1) P_{LM} (x^T | x^{T-1}, \dots, x^1) P_{LM} (x^{T-1}, \dots, x^1) \\ &= \prod_{t=1}^T P_{LM} (x^{t+1} | x^t, \dots, x^1) \end{aligned}$$

Ejemplo:

$$P(\text{"La hojarasca crepitar"}) = P(\text{"crepitar"} | \text{"La hojarasca"}) P(\text{"hojarasca"} | \text{"la"}) P(\text{"la"})$$